# PROPOSAL PENELITIAN

# RANCANG BANGUN SISTEM SPEECH TO TEXT BERBASIS WEB MENGGUNAKAN MODEL OPEN AI WHISPER

UIN IMAM BONJOL
PADANG

**Oleh:**

**MUHAMAD IFFAN RAMADHAN**

**NIM. 2117020036**

**PROGRAM STUDI SISTEM INFORMASI**
**FAKULTAS SAINS DAN TEKNOLOGI**
**UNIVERSITAS ISLAM NEGERI IMAM BONJOL PADANG**
**2026**

# PROPOSAL PENELITIAN

# RANCANG BANGUN SISTEM SPEECH TO TEXT BERBASIS WEB MENGGUNAKAN MODEL OPEN AI WHISPER

Diajukan Sebagai Salah Satu Syarat untuk Meraih Gelar Sarjana (S1)

UIN IMAM BONJOL
PADANG


**Oleh:**

**MUHAMAD IFFAN RAMADHAN**

**NIM. 2117020036**



**PROGRAM STUDI SISTEM INFORMASI**
**FAKULTAS SAINS DAN TEKNOLOGI**
**UNIVERSITAS ISLAM NEGERI IMAM BONJOL PADANG**


**2026**

## ABSTRAK

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.



**Kata Kunci:** Transkripsi, Whisper, SaaS, Benchmarking.

# HALAMAN PERSETUJUAN

Proposal penelitian dengan judul **"Rancang Bangun Sistem Speech To Text Berbasis Web Menggunakan Model Open AI Whisper"** yang disusun oleh MUHAMAD IFFAN RAMADHAN NIM 2117020036, telah diperiksa dan disetujui untuk diajukan dalam seminar proposal pada Program Studi Sistem Informasi Fakultas Sains dan Teknologi Universitas Islam Negeri Imam Bonjol Padang.

Padang, ..... 2026

Menyetujui:

| Pembimbing I | Pembimbing II |
| --- | --- |
| **Ahmad Fauzi, M.T.I** | **Ozzy Secio Riza, M.Kom** |
| NIP. 123456789 | NIP. 987654321 |

## DAFTAR ISI


**ABSTRAK** ..... **iii**

**HALAMAN PERSETUJUAN** ..... **iv**

**DAFTAR TABEL** ..... **vi**

**DAFTAR GAMBAR** ..... **vii**

**BAB I PENDAHULUAN** ..... **1**

A Latar Belakang ..... 1

B Rumusan Masalah ..... 3

C Batasan Masalah ..... 3

D Tujuan Penelitian ..... 3

E Manfaat Penelitian ..... 4

**BAB II TINJAUAN PUSTAKA** ..... **5**

A Kerangka Teori ..... 5

B Literatur Review ..... 14

**BAB IIIMETODE PENELITIAN** ..... **20**

A Metode Penelitian ..... 20

B Tempat Penelitian ..... 20

C Prosedur Penelitian ..... 20

D Data dan Sumber Data ..... 23

E Alat dan Instrumen Penelitian ..... 24

F Jadwal Penelitian ..... 25

**BAB IVHASIL DAN PEMBAHASAN** ..... **26**

A Hasil Penelitian ..... 26

B Pembahasan ..... 26

**BAB V PENUTUP** ..... **27**

A Kesimpulan ..... 27

B Saran ..... 27

**DAFTAR PUSTAKA** ..... **28**

## DAFTAR TABEL

2.1 Ringkasan Literatur Review ..... 15
3.1 Statistik Dataset Penelitian ..... 24
3.2 Jadwal Penelitian (*Gantt Chart*) ..... 25

## DAFTAR GAMBAR

3.1 Alur Penelitian (*Flowchart*)..... 23

# BAB I
# PENDAHULUAN

## A. Latar Belakang

Proses mengubah data suara menjadi teks (*transkripsi*) merupakan tahapan yang krusial namun seringkali menjadi beban kerja terberat dalam dokumentasi informasi, terutama pada konten berdurasi panjang seperti siniar (*podcast*) atau rekaman wawancara penelitian. Meskipun data audio sangat kaya akan informasi, keterbatasannya dalam hal pencarian kata kunci dan aksesibilitas membuat transkripsi teks menjadi kebutuhan yang mendesak bagi para peneliti, jurnalis, dan institusi pendidikan (Imai, Chowdhury, and Stent, n.d.). Masalah utamanya adalah transkripsi manual memerlukan konsentrasi tinggi dan waktu yang sangat lama, di mana seorang transkriptor profesional sekalipun biasanya membutuhkan waktu hingga lima jam hanya untuk menyelesaikan satu jam rekaman audio (Christodoulidou , dkk., 2025). Di berbagai instansi dan lembaga pendidikan, keterbatasan waktu ini seringkali menyebabkan banyak data audio berharga berakhir menjadi tumpukan berkas digital yang sulit dimanfaatkan secara optimal.

Perkembangan teknologi *Automatic Speech Recognition* (ASR) berbasis *Transformer*, seperti model *Whisper* yang dikembangkan oleh OpenAI, menawarkan potensi besar untuk memangkas waktu kerja tersebut melalui otomatisasi. Dengan pelatihan menggunakan 680.000 jam data audio multibahasa, *Whisper* mampu mengenali berbagai dialek dan istilah teknis dengan tingkat akurasi yang melampaui generasi model ASR sebelumnya (Radford , dkk., n.d.). Namun, efektivitas model ini pada data percakapan alami bahasa Indonesia masih menghadapi tantangan berupa ambiguitas linguistik, gangguan latar belakang, dan penggunaan bahasa gaul yang sering muncul dalam siniar. Oleh karena itu, hasil transkripsi otomatis belum dapat dianggap sebagai produk akhir yang sempurna dan tetap membutuhkan proses penyuntingan manual untuk menjamin kualitasnya. Penelitian sebelumnya menunjukkan bahwa alur kerja yang menggabungkan draf otomatis dengan antarmuka koreksi manusia (*human-in-the-loop*) dapat meningkatkan produktivitas

secara signifikan jika dikelola melalui sistem yang efisien (Olev and Alumäe, 2022).

Tantangan berikutnya muncul dari sisi infrastruktur, di mana penggunaan layanan ASR berbasis *cloud* komersial seringkali terbentur pada masalah biaya yang mahal dan kekhawatiran terkait keamanan data sensitif (Ferraro, Galli, La Gatta, and Postiglione, 2023). Bagi institusi lokal, mengirimkan data audio internal ke server pihak ketiga di luar negeri bukan hanya membebani anggaran, tetapi juga berisiko terhadap privasi informasi yang terkandung di dalamnya. Pemanfaatan perangkat keras internal dengan spesifikasi *Graphic Processing Unit* (GPU) mumpuni yang sudah tersedia di berbagai komputer atau laptop operasional instansi dapat menjadi solusi alternatif yang lebih aman dan berkelanjutan untuk menjalankan model AI secara mandiri (*self-hosted*). Akan tetapi, pengoperasian model AI skala besar secara lokal membutuhkan sistem manajemen tugas yang kokoh agar sumber daya komputasi dapat digunakan secara optimal tanpa mengganggu kinerja aplikasi bagi pengguna. Kemandirian teknologi dalam pengolahan data bahasa ini menjadi sangat penting untuk memastikan keberlanjutan operasional sistem di masa depan (Fathony, Mareta, Adiana, Wardhani, and Halim, n.d.).

Sistem Skriptor hadir sebagai solusi untuk menjembatani kesenjangan antara kecanggihan model AI dengan kebutuhan praktis pengguna di lapangan. Skriptor dirancang sebagai platform transkripsi mandiri yang menggunakan arsitektur terdistribusi dengan memisahkan antarmuka pengguna berbasis *Next.js* dari unit pemrosesan AI berbasis *Python Worker*. Pendekatan ini memungkinkan sistem untuk tetap responsif dan stabil meskipun sedang menangani beban kerja inferensi yang berat pada berkas audio berdurasi panjang. Melalui arsitektur ini, pengguna dapat melakukan koreksi teks secara interaktif sementara sistem terus memproses antrean audio lainnya di latar belakang secara asinkron. Penelitian ini berfokus pada rancang bangun sistem *speech-to-text* berbasis *web* dengan mengintegrasikan model OpenAI *Whisper* guna meningkatkan produktivitas alur kerja transkripsi audio bahasa Indonesia.

## B. Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana rancang bangun sistem *speech-to-text* berbasis *web* dengan arsitektur terdistribusi menggunakan *Next.js* dan *Python Worker*?
2. Bagaimana performa kecepatan (*inference time*), konsumsi VRAM, dan tingkat akurasi (*Word Error Rate* dan *Character Error Rate*) dari berbagai varian model OpenAI *Whisper* ketika diintegrasikan ke dalam sistem tersebut?
3. Bagaimana tingkat *usability* antarmuka sistem *speech-to-text* berbasis *web* yang dikembangkan berdasarkan *System Usability Scale* (SUS)?

## C. Batasan Masalah

Penelitian ini dibatasi pada hal-hal sebagai berikut:

1. Model yang diuji terbatas pada varian *Whisper* (*Tiny, Small, Medium, Large-v3, Large-v3-Turbo*).
2. Sistem dibangun menggunakan framework *Next.js* 16 dengan dukungan *TypeScript* dan arsitektur *distributed task queue* berbasis *Redis* untuk menjamin skalabilitas.
3. Dataset yang digunakan berasal dari korpus *Mozilla Common Voice* (Siniar *Homostoria* dan *Hari Minggoean*) dalam bahasa Indonesia.
4. Infrastruktur pengujian menggunakan spesifikasi NVIDIA RTX A5000 pada lingkungan *Runpod*.

## D. Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Merancang dan membangun sistem *speech-to-text* berbasis *web* dengan arsitektur terdistribusi menggunakan *Next.js* dan *Python Worker*.
2. Mengevaluasi performa kecepatan, konsumsi VRAM, dan tingkat akurasi (WER dan CER) dari berbagai varian model OpenAI *Whisper* di dalam sistem terdistribusi yang dikembangkan.

3. Mengukur tingkat *usability* antarmuka sistem *speech-to-text* berbasis *web* yang dikembangkan menggunakan kuesioner *System Usability Scale* (SUS).

## E. Manfaat Penelitian

Manfaat yang diharapkan dari penelitian ini adalah:

1. **Manfaat Teoritis:** Memberikan data *benchmarking* terbaru mengenai performa model ASR *Transformer* pada bahasa Indonesia serta kontribusi pada literatur rancang bangun sistem AI terdistribusi. Penelitian ini juga memperkaya kajian mengenai interaksi manusia dan komputer dalam proses koreksi data hasil otomatisasi.
2. **Manfaat Praktis:** Menyediakan solusi sistem transkripsi mandiri (*self-hosted*) bernama Skriptor yang ekonomis bagi institusi akademis dan profesional. Sistem ini membantu mengurangi biaya operasional, meningkatkan keamanan privasi data, dan mempercepat alur kerja pembuatan transkrip melalui fitur penyuntingan yang terintegrasi.

# BAB II
# TINJAUAN PUSTAKA

## A. Kerangka Teori

Bagian ini memaparkan teori-teori dasar yang menjadi landasan dalam pengembangan dan analisis sistem transkripsi terdistribusi.

### 1. Automatic Speech Recognition (ASR)

*Automatic Speech Recognition* (ASR) adalah teknologi yang memungkinkan komputer untuk mengubah sinyal suara menjadi teks secara otomatis. Proses ini melibatkan konversi gelombang akustik mentah menjadi representasi digital yang kemudian dipetakan ke dalam satuan bahasa (fonem, suku kata, atau kata). Secara tradisional, sistem ASR mengandalkan komponen terpisah seperti model akustik, model bahasa, dan leksikon pelafalan. Namun, perkembangan terkini telah bergeser ke arah model *End-to-End* (E2E) yang menyederhanakan alur kerja tersebut menjadi satu jaringan saraf tunggal yang melakukan pemetaan langsung dari audio ke teks (Prabhavalkar, Hori, Sainath, Schlüter, and Watanabe, 2023).

### 2. Arsitektur Model Whisper (OpenAI)

Model *Whisper* yang dikembangkan oleh OpenAI merupakan arsitektur jaringan saraf *End-to-End* (E2E) berbasis *Transformer Encoder-Decoder* yang dilatih menggunakan pendekatan *weakly supervised learning* pada skala data sebesar 680.000 jam audio multibahasa (Radford , dkk., n.d.). Secara detail, arsitektur model ini terdiri dari tiga komponen utama: pemrosesan awal audio, *encoder*, dan *decoder* autoregresif.

1. **Pra-pemrosesan Audio (*Audio Pre-processing*):** Sinyal audio mentah (*raw audio*) pertama-tama diubah formatnya menjadi berfrekuensi tunggal (*monophonic*) dengan laju sampel (*sampling rate*) sebesar 16.000 Hz. Gelombang audio kontinu ini kemudian dipecah menjadi potongan-potongan (*chunks*) statis berdurasi 30 detik. Pemrosesan awal menggunakan

transformasi Fourier jangka pendek (*Short-Time Fourier Transform* atau STFT) dengan jendela Hamming berukuran 25 milidetik dan pergeseran (*stride/hop length*) sebesar 10 milidetik. Hasil ekstraksi berupa spektrogram log-Mel dengan 80 dimensi (atau 128 dimensi pada varian *Whisper v3*) yang dinormalisasi ke rentang $[-1, 1]$. Spektrogram log-Mel ini direpresentasikan sebagai matriks input $\mathbf{X} \in \mathbb{R}^{F \times T}$, di mana $F$ adalah jumlah filter Mel (80 atau 128) dan $T$ adalah jumlah bingkai waktu (*time frames*) yang bernilai 3000 bingkai untuk 30 detik audio.

2. **Arsitektur *Encoder* Transformer:** *Encoder* bertugas memetakan spektrogram input menjadi representasi fitur abstrak yang kaya akan informasi akustik. Pertama, spektrogram $\mathbf{X}$ dilewatkan ke dalam blok konvolusi yang terdiri dari dua lapisan konvolusi 1D dengan lebar filter (*filter width*) sebesar 3, fungsi aktivasi *Gaussian Error Linear Unit* (GELU), dan pergeseran (*stride*) sebesar 2 pada sumbu temporal. Proses ini secara matematis mereduksi dimensi bingkai waktu dari $T$ menjadi $T' = \lfloor \frac{T}{4} \rfloor = 750$. Penurunan dimensi ini sangat penting untuk mengurangi beban komputasi kuadratis dari mekanisme perhatian pada *Transformer*. Setelah itu, ditambahkan *sinusoidal positional embeddings* untuk menyuntikkan informasi urutan temporal ke dalam representasi audio. Sinyal yang telah tereduksi dan tertambah posisi ini dimasukkan ke dalam tumpukan blok *Transformer Encoder*. Setiap blok memiliki struktur standar *Multi-Head Self-Attention* (MHA) dan jaringan saraf *Feed-Forward* (FFN) dua lapis dengan fungsi aktivasi GELU. Sebelum masuk ke MHA dan FFN, diterapkan normalisasi lapisan (*Layer Normalization*) dengan pendekatan *Pre-Activation Layer Normalization* (Pre-LN) untuk menjaga stabilitas aliran gradien pada arsitektur yang sangat dalam:

$$\mathbf{Z}^{(l-1)*} = \mathbf{Z}^{(l-1)} + \mathrm{MHA}(\mathrm{LN}(\mathbf{Z}^{(l-1)})) \tag{2.1}$$

$$\mathbf{Z}^{(l)} = \mathbf{Z}^{(l-1)*} + \mathrm{FFN}(\mathrm{LN}(\mathbf{Z}^{(l-1)*})) \tag{2.2}$$

di mana $\mathbf{Z}^{(l)}$ adalah representasi fitur pada lapisan ke-$l$.

3. **Arsitektur *Decoder* Transformer:** *Decoder* berfungsi sebagai model bahasa kondisional (*conditional language model*) yang memprediksi urutan token teks secara autoregresif berdasarkan representasi akustik yang dihasilkan oleh *encoder*. Blok *decoder* menggunakan mekanisme *masked Multi-Head Self-Attention* untuk mencegah model mengamati token teks di masa depan (*look-ahead bias*) selama proses inferensi sekuensial. Selain itu, ditambahkan mekanisme *Multi-Head Cross-Attention* yang menghubungkan tumpukan *decoder* dengan output representasi akustik dari *encoder* ($\mathbf{Z}^{(L)}$), yang secara matematis dirumuskan sebagai:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.3}$$

di mana matriks *query* ($Q$) diproyeksikan dari status representasi token teks sebelumnya, sedangkan *key* ($K$) dan *value* ($V$) diproyeksikan dari representasi akustik $\mathbf{Z}^{(L)}$ milik *encoder*. Proyeksi akhir dari *decoder* dipetakan kembali ke ruang kosakata teks (*vocabulary space*) menggunakan fungsi *softmax* untuk menghasilkan distribusi probabilitas token teks berikutnya.

4. *Multitask dan Multilingual Format* (Format Token Khusus): Model *Whisper* dapat mengintegrasikan berbagai tugas pemrosesan suara ke dalam satu arsitektur terpadu dengan memanfaatkan rangkaian token kontrol khusus (*special tokens*) yang diposisikan di awal urutan *decoder*. Rangkaian token tersebut diilustrasikan sebagai berikut:
    - $\langle|$**startoftranscript**$|\rangle$: Token penanda mulainya proses dekoding teks.
    - **Token Bahasa**: Menentukan bahasa sasaran (misalnya $\langle|$id$|\rangle$ untuk Bahasa Indonesia atau $\langle|$en$|\rangle$ untuk Bahasa Inggris). Whisper mendukung identifikasi bahasa secara otomatis (*Language Identification*) dengan memprediksi token bahasa dengan probabilitas tertinggi.
    - **Token Tugas**: Menentukan fungsi yang akan dijalankan oleh

model, yaitu ⟨|transcribe|⟩ untuk transkripsi pidato ke teks asli, atau ⟨|translate|⟩ untuk menerjemahkan teks ucapan secara langsung ke Bahasa Inggris.

- **Token Kontrol Waktu**: Menentukan apakah model harus menghasilkan tanda waktu pada tingkat kalimat (⟨|notimestamps|⟩ dilewati) atau mengabaikan pembuatan tanda waktu untuk meningkatkan kecepatan komputasi.

### 3. Inovasi *Pipeline* WhisperX

Meskipun *Whisper* memiliki kemampuan generalisasi yang sangat baik, model ini menghadapi keterbatasan serius ketika diterapkan pada audio berdurasi panjang, seperti masalah pergeseran tanda waktu (*timestamp drift*), halusinasi teks pada area hening (*hallucination loops*), dan latensi komputasi yang tinggi akibat sifat pemrosesan sekuensial autoregresif. Untuk mengatasi masalah tersebut, Bain, Huh, Han, and Zisserman (2023) memperkenalkan *WhisperX*, sebuah kerangka kerja terpadu berkinerja tinggi yang menggabungkan deteksi aktivitas suara, inferensi berkelompok, dan penyelarasan fonem paksa.

1. ***Voice Activity Detection* (VAD) *Cut & Merge*:** Sebelum audio dikirim ke model ASR, *WhisperX* menerapkan pra-pemrosesan berupa deteksi aktivitas suara (VAD) menggunakan model saraf terlatih (seperti *Silero VAD* atau *PyAnnote VAD*). Algoritma VAD memisahkan sinyal audio kontinu menjadi segmen-segmen bicara aktif dan menghilangkan bagian hening atau non-vokal (seperti musik latar atau kebisingan). Segmen-segmen aktif ini kemudian digabungkan secara dinamis menggunakan metode *Cut & Merge* hingga mencapai panjang optimal berdurasi maksimal 30 detik. Dengan memastikan pemotongan segmen terjadi tepat pada fase hening alami (di antara kata atau kalimat), *WhisperX* mengeliminasi kecenderungan *Whisper* asli melakukan pengulangan frasa secara berulang (*repetition loops*) saat menghadapi jeda hening yang panjang.
2. ***Batched Transcription* via *Faster-Whisper*:** *Whisper* asli memproses

audio secara berurutan (*sequential decoding*), yang mengharuskan hasil transkripsi dari segmen sebelumnya digunakan sebagai konteks untuk segmen berikutnya. Hal ini menghalangi paralelisasi komputasi pada GPU. Karena pembagian segmen audio berbasis VAD pada *WhisperX* menghasilkan batas potongan yang bersih, setiap segmen berdurasi 30 detik dapat ditranskripsikan secara mandiri dan paralel dalam satu kelompok (*batch*). Untuk mencapai performa maksimal, *WhisperX* mengintegrasikan pustaka *Faster-Whisper* yang merupakan reimplementasi model *Whisper* dalam *C++* menggunakan pustaka optimasi inferensi *CTranslate2*. Kombinasi *batching* dan optimasi ini mempercepat waktu pemrosesan (*throughput*) hingga 12 kali lipat pada arsitektur GPU NVIDIA modern.

3. ***Forced Phoneme Alignment* (Alineasi Paksa):** Penanda waktu bawaan *Whisper* asli diprediksi pada tingkat frasa (berdasarkan token koordinat waktu dalam rangkaian dekoding) dengan presisi kasar (resolusi 20 ms) dan rentan terhadap pergeseran temporal (*drifting*). Untuk menghasilkan penanda waktu tingkat kata (*word-level timestamp*) yang akurat, *WhisperX* menerapkan tahap pasca-pemrosesan berupa *forced alignment* menggunakan model pengenalan suara berbasis koneksi klasifikasi temporal (*Connectionist Temporal Classification* atau CTC), seperti *Wav2Vec 2.0* atau *WavLM* yang sesuai dengan bahasa target. Algoritma *forced alignment* bekerja dengan langkah-langkah berikut:
    - Teks transkripsi hasil dari dekoding Whisper dikonversi menjadi urutan fonem standar.
    - Audio dilewatkan ke model CTC untuk memprediksi probabilitas kemunculan karakter/fonem pada setiap bingkai waktu audio.
    - Algoritma Viterbi digunakan untuk menghitung jalur penyelarasan optimal (*optimal alignment path*) yang memetakan urutan fonem dari teks ke koordinat bingkai waktu sinyal akustik. Hasil pemetaan ini memberikan koordinat batas waktu awal dan akhir ($t_{start}, t_{end}$) untuk setiap kata secara akurat dengan tingkat kesalahan di bawah 50

milidetik.

4. ***Speaker Diarization* (Diarisasi Pembicara):** Tahap terakhir dalam *pipeline WhisperX* adalah proses klasifikasi pembicara untuk menentukan identitas suara dari setiap kata yang diucapkan (*"who spoke when"*). Proses ini memanfaatkan pustaka *PyAnnote Audio* yang terdiri dari tiga langkah:
    - **Ekstraksi *Embeddings* Pembicara**: Menggunakan arsitektur jaringan saraf *ECAPA-TDNN* untuk mengekstraksi vektor representasi suara (*speaker embeddings*) berdimensi tinggi dari segmen audio.
    - ***Spectral Clustering***: Melakukan pengklasteran spektral terhadap vektor-vektor *embeddings* untuk memisahkan dan mengelompokkan suara ke dalam kelas unik yang merepresentasikan identitas pembicara yang berbeda (misalnya Pembicara 1, Pembicara 2).
    - **Resolusi *Word-Level***: Dengan mengandalkan koordinat waktu tingkat kata yang sangat presisi hasil dari tahap *forced alignment*, identitas pembicara dipetakan langsung ke setiap kata dalam transkrip akhir. Langkah ini meminimalisir kesalahan pelabelan pembicara pada ucapan tumpang tindih (*overlapped speech*) atau jeda bicara yang sangat rapat.

### 4. Optimasi Inferensi: Faster-Whisper dan Kuantisasi

Untuk menjalankan model besar pada lingkungan produksi yang efisien, diperlukan teknik optimasi inferensi.

1. ***CTranslate2*:** Pustaka yang mengimplementasikan inferensi model *Transformer* secara efisien melalui teknik penggabungan operator (*operator fusion*) dan manajemen memori yang ketat.
2. **Kuantisasi Model:** Proses pengurangan presisi numerik bobot model (misalnya dari FP16 ke INT8). Teknik ini secara signifikan mereduksi penggunaan VRAM dan mempercepat waktu eksekusi dengan degradasi akurasi yang minimal (Andreyev, 2025).
3. ***Real Time Factor* (RTF):** Metrik utama yang mengukur kecepatan inferensi

dengan membandingkan waktu pemrosesan terhadap durasi audio asli. RTF < 1 menunjukkan sistem mampu memproses audio lebih cepat dari waktu nyata.

### 5. Metrik Evaluasi: WER dan CER

Akurasi sistem ASR diukur menggunakan metrik berbasis *Levenshtein Distance*.

1. ***Word Error Rate* (WER):** Menghitung rasio kesalahan pada tingkat kata (substitusi, penghapusan, penyisipan) terhadap total kata referensi (Ferraro , dkk., 2023).
2. ***Character Error Rate* (CER):** Serupa dengan WER namun diterapkan pada tingkat karakter, yang sangat berguna untuk mengevaluasi ketelitian ejaan pada bahasa dengan morfologi kompleks.

### 6. Arsitektur Sistem Terdistribusi

Sistem skala besar memerlukan pemisahan beban kerja melalui arsitektur terdistribusi.

1. ***Message Queue Pattern*:** Pola arsitektur di mana produsen tugas (*producer*) memasukkan pekerjaan ke dalam antrean (*queue*) yang kemudian diambil oleh unit pemrosesan (*worker*) secara asinkron.
2. ***Worker Pattern*:** Memungkinkan skalabilitas horizontal di mana beberapa unit pemrosesan dapat berjalan secara paralel untuk menangani antrean tugas yang intensif komputasi (Macháček, Dabre, and Bojar, 2023).

### 7. Human-in-the-Loop (HITL), *Usability*, dan *System Usability Scale*

Konsep *Human-in-the-Loop* (HITL) mengakui bahwa sistem kecerdasan buatan, secanggih apa pun, tidak dapat sepenuhnya menggantikan penilaian manusia dalam tugas-tugas yang memerlukan pemahaman kontekstual dan ketelitian tinggi. Dalam konteks transkripsi, HITL menempatkan manusia sebagai validator akhir

yang bertugas memeriksa dan mengoreksi hasil keluaran model ASR sebelum draf transkripsi dianggap final. Penelitian terdahulu menunjukkan bahwa alur kerja semi-otomatis yang menggabungkan draf transkripsi dari AI dengan antarmuka penyuntingan yang intuitif dapat meningkatkan produktivitas secara signifikan, terutama jika antarmuka tersebut dilengkapi dengan fitur visualisasi gelombang audio yang tersinkronisasi dengan teks (Imai , dkk.; Olev and Alumäe, n.d.; 2022). Namun, efektivitas pendekatan HITL sangat bergantung pada tingkat kegunaan (*usability*) antarmuka yang disediakan bagi pengguna.

Menurut standar internasional ISO 9241-11, *usability* didefinisikan sebagai sejauh mana suatu produk dapat digunakan oleh pengguna tertentu untuk mencapai tujuan tertentu dengan efektivitas (*effectiveness*), efisiensi (*efficiency*), dan kepuasan (*satisfaction*) dalam konteks penggunaan tertentu. Untuk mengukur aspek *usability* secara kuantitatif pada sistem yang dikembangkan, diperlukan instrumen pengukuran yang terstandarisasi dan reliabel.

*System Usability Scale* (SUS) yang dikembangkan oleh Brooke (1996) merupakan kuesioner terstandarisasi yang terdiri dari 10 butir pernyataan dengan skala Likert 5 poin (1 = Sangat Tidak Setuju hingga 5 = Sangat Setuju). Kuesioner ini dirancang untuk memberikan penilaian global terhadap kegunaan suatu sistem secara cepat dan murah. Sepuluh butir pernyataan SUS disusun secara bergantian antara pernyataan bernada positif (butir ganjil: 1, 3, 5, 7, 9) dan pernyataan bernada negatif (butir genap: 2, 4, 6, 8, 10). Prosedur perhitungan skor SUS adalah sebagai berikut:

1. Untuk setiap butir bernada positif (ganjil), kontribusi skor dihitung dengan mengurangi 1 dari respons pengguna: $(x_i - 1)$.
2. Untuk setiap butir bernada negatif (genap), kontribusi skor dihitung dengan mengurangi respons pengguna dari 5: $(5 - x_i)$.
3. Jumlahkan seluruh kontribusi skor dari 10 butir, lalu kalikan dengan 2,5 untuk mendapatkan skor akhir dalam rentang 0–100.

Secara matematis, rumus skor SUS dapat dituliskan sebagai:

$$\text{SUS} = 2{,}5 \times \left[ \sum_{i \in \text{ganjil}} (x_i - 1) + \sum_{j \in \text{genap}} (5 - x_j) \right] \tag{2.4}$$

Berdasarkan studi empiris yang dilakukan oleh Bangor, Kortum, and Miller (2008) terhadap lebih dari 2.000 survei, interpretasi skor SUS dikategorikan ke dalam tiga tingkatan penerimaan: skor di atas 70 dikategorikan sebagai *Acceptable* (dapat diterima), skor antara 50–70 dikategorikan sebagai *Marginal* (perlu perbaikan), dan skor di bawah 50 dikategorikan sebagai *Not Acceptable* (tidak dapat diterima). Selain itu, Bangor , dkk. juga memetakan skor SUS ke dalam skala penilaian deskriptif (*adjective rating*): *Excellent* ($\geq$ 85,5), *Good* (72,6–85,5), *OK* (50,9–72,5), *Poor* (25,1–50,9), dan *Worst Imaginable* ($\leq$ 25,0).

### 8. Pengujian Perangkat Lunak

Pengujian perangkat lunak (*software testing*) merupakan proses evaluasi terhadap suatu sistem untuk memastikan bahwa perilaku dan kinerjanya telah sesuai dengan spesifikasi yang diharapkan serta terbebas dari kesalahan (*bugs* atau *errors*). Pengujian ini bertujuan untuk meningkatkan keandalan, keamanan, dan kualitas perangkat lunak secara keseluruhan sebelum dirilis ke lingkungan produksi. Secara garis besar, pengujian perangkat lunak dibagi menjadi beberapa metode dan tingkatan, antara lain:

1. ***Black-box Testing* (Pengujian Kotak Hitam):** Pengujian yang berfokus pada evaluasi fungsionalitas sistem tanpa harus mengetahui struktur kode internal atau logika program di dalamnya. Pengujian ini dilakukan dengan memberikan *input* tertentu ke sistem dan memverifikasi apakah *output* yang dihasilkan telah sesuai dengan spesifikasi kebutuhan pengguna.
2. **Pengujian Unit (*Unit Testing*):** Pengujian pada tingkat terkecil dari komponen perangkat lunak (seperti fungsi, kelas, atau modul individual) untuk memastikan bahwa setiap unit kode tersebut berfungsi dengan benar secara terisolasi.

3. **Pengujian Integrasi (*Integration Testing*):** Pengujian yang memverifikasi interaksi dan aliran data antara beberapa modul atau komponen sistem yang saling terhubung untuk memastikan mereka dapat bekerja sama dengan baik.
4. **Pengujian Otomatis (*Automated Testing*):** Proses pengujian yang dijalankan secara otomatis menggunakan perangkat lunak khusus untuk mengeksekusi skenario pengujian yang telah diprogram sebelumnya. Pengujian otomatis sangat efektif untuk mempercepat proses verifikasi kode dan mencegah terjadinya penurunan kualitas (*regression*) saat kode diperbarui. Salah satu *framework* pengujian otomatis yang populer di ekosistem *Javascript* modern adalah *Vitest*.

### 9. Karakteristik Bahasa Indonesia untuk ASR

Bahasa Indonesia memiliki tantangan tersendiri dalam ASR karena variasi dialek dan statusnya yang sering dikategorikan sebagai *low-resource language* dalam beberapa korpus besar. Meskipun Whisper memiliki performa multibahasa yang baik, akurasinya pada bahasa Indonesia sering kali lebih rendah dibandingkan bahasa Inggris, sehingga memerlukan evaluasi spesifik menggunakan *dataset* lokal (Fathony , dkk., n.d.).

## B. Literatur Review

Tinjauan pustaka dilakukan dengan menganalisis berbagai penelitian terdahulu yang memiliki relevansi dengan pemrosesan bahasa alami dan arsitektur sistem terdistribusi. Data literatur tersebut dirangkum dalam Tabel 2.1 untuk memberikan gambaran komprehensif mengenai metode, tujuan, dan temuan utama dari setiap penelitian yang dirujuk.

**Tabel 2.1:** Ringkasan Literatur Review

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| User-centered Design Methods in Data-Intensive Software... | Riebel (2023) | *Literature Review* | Menganalisis integrasi UCD dalam AI | UCD meningkatkan empati dan definisi masalah dalam pengembangan AI. |
| Benchmarking open source and paid services for speech to text... | Ferraro , dkk. (2023) | *Benchmark & WER Analysis* | Membandingkan performa layanan ASR berbayar dan open-source. | Layanan berbayar lebih akurat, namun performa sangat bergantung pada variasi input teks. |
| Evaluating Open-Source ASR Systems... | Imai , dkk. (2025) | Evaluasi Eksperimental | Mengukur akurasi ASR pada audio bising | Perlu *robust error correction* untuk mengatasi bias demografis. |
| End-to-End Speech Recognition: A Survey | Prabhavalkar , dkk. (2023) | *Survey* Literatur | Memetakan perkembangan E2E ASR | Model E2E mereduksi WER hingga 50% dibanding HMM tradisional. |
| Transformers in Speech Processing: A Survey | Latif , dkk. (2025) | *Survey* Literatur | Menganalisis *Transformer* dalam audio | *Transformer* unggul dalam memodelkan dependensi jangka panjang. |
| Berlanjut ke halaman berikutnya | | | | |

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| Multilingual DistilWhisper... | Ferraz, Boito, Brun, and Nikoulina (2024) | Eksperimental (*Distillation*) | Meringankan model Whisper untuk bahasa langka | *DistilWhisper* meningkatkan performa bahasa *under-represented* dengan *overhead* minim. |
| Does ChatGPT and Whisper Make Humanoid Robots More Relatable? | Chen, Luo, Gee, and Nejati (2024) | Evaluasi Pengguna | Membandingkan Whisper vs Google ASR pada robot | Whisper memiliki WER terendah (1.71%) dan paling disukai oleh pengguna. |
| Quantization for OpenAI's Whisper Models... | Andreyev (2025) | Analisis Komparatif | Mengukur dampak kuantisasi pada latensi | Kuantisasi INT8 mereduksi size 45% dan latensi 19% tanpa degradasi akurasi. |
| A Self-Training Approach for Whisper to Enhance Long Dysarthric Speech... | Wang, Zhou, Zhao, and Qin (2025) | *Self-training method* | Meningkatkan ASR untuk penderita disartria | Metode *self-training* meningkatkan performa pada ucapan penderita disabilitas. |
| Berlanjut ke halaman berikutnya | | | | |

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| Accurate Speech Recognition AI using Model of Deep Learning... | Isyanto, Ibrahim, Samsinar, and Sudarwati (2024) | Eksperimental CNN | Membangun ASR untuk keamanan perbankan | Algoritma CNN mencapai akurasi 99% untuk pengenalan kata kunci keamanan. |
| Evaluating OpenAI's Whisper ASR... | Graham and Roll (2024) | Analisis Performa | Mengevaluasi Whisper pada berbagai aksen | Whisper lebih akurat pada aksen asli (*native*) dibandingkan aksen *non-native*. |
| The Role of TypeScript in Enhancing Development... | Sekhar Emmann (2021) | *Empirical Analysis* | Menganalisis peran TypeScript dalam pengembangan aplikasi *web* modern. | TypeScript secara signifikan mengurangi kesalahan *runtime* dan meningkatkan skalabilitas. |
| A Comparative Study of Agile and Waterfall Software Development... | Justin Joseph John and Shailesh Satish Sharma (2024) | Analisis Komparatif | Membandingkan *Agile* vs *Waterfall* | *Agile* lebih unggul dalam kepuasan pelanggan dan kecepatan pengembangan. |
| Berlanjut ke halaman berikutnya | | | | |

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| Turning Whisper into Real-Time Transcription System | Macháček , dkk. (2023) | Implementasi | Mengembangkan sistem transkripsi *real-time* berbasis Whisper. | *Whisper-Streaming* mencapai latensi 3,3 detik dengan akurasi yang tetap terjaga. |
| WhisperX: Time-Accurate Speech Transcription... | Bain , dkk. (2023) | Pengembangan Sistem | Penyelarasan waktu tingkat kata pada Whisper | VAD dan *forced alignment* memungkinkan transkripsi 12x lebih cepat. |
| Bridging the Gap Between Generalized ASR and Domain-Specific... | Nelson and Gretkzi (2023) | Integrasi Vosk | Menyesuaikan ASR untuk domain spesifik | Language modeling pada Vosk efektif untuk istilah medis dan hukum. |
| A semi-automatic pipeline for transcribing child speech | ChristodoulidouEvaluasi , dkk. (2025) | *Pipeline* | Membangun *pipeline* transkripsi suara anak | Koreksi manual sebelum *forced alignment* meningkatkan kualitas data akustik. |
| Comparing ASR Systems in Context of Speech Disfluencies | Teleki, Dong, Kim, and Caverlee (2024) | Analisis Korpus Besar | Mengevaluasi Google ASR vs Whisper | Whisper lebih unggul dalam menangani interjeksi dan keraguan bicara. |

Berlanjut ke halaman berikutnya

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| Perancangan Antarmuka Pengguna Aplikasi Informasi Siswa... | Pratiwi, Wijoyo, and Wicaksono (2022) | *User Centered Design* (UCD) | Merancang aplikasi informasi sekolah | Metode UCD menghasilkan desain yang 100% dapat digunakan oleh target *user*. |
| Estonian Speech Recognition and Transcription Editing Service | Olev and Alumäe (2022) | Pengembangan Layanan | Membangun sistem transkripsi berbasis wav2vec2.0 | Sistem berbasis E2E mencapai WER 6.9% dan mendukung *editing* kolaboratif. |

Berdasarkan tinjauan literatur di atas, penelitian ini memfokuskan diri pada integrasi sistem transkripsi berbasis *Next.js* dan *Python Worker* dalam arsitektur terdistribusi yang dioptimasi untuk penggunaan institusi lokal. Penelitian ini mengisi celah dengan menggabungkan performa model WhisperX yang telah terbukti unggul dengan antarmuka koreksi manual yang dirancang menggunakan prinsip UCD. Melalui pendekatan ini, diharapkan transkripsi bahasa Indonesia pada data siniar dapat dilakukan dengan akurasi yang lebih tinggi dan waktu koreksi yang lebih singkat.

# BAB III
# METODE PENELITIAN

## A. Metode Penelitian

Penelitian ini dikategorikan sebagai penelitian rekayasa perangkat lunak dengan metode pengembangan sistem (*Research and Development* atau R&D). Pengembangan sistem ini mengikuti tahapan perancangan, implementasi, dan pengujian. Pengujian sistem dilakukan dalam tiga aspek utama: pengujian fungsionalitas sistem secara langsung (*black-box testing*) serta secara otomatis (*automated unit and integration testing*) untuk memverifikasi keandalan kode, pengujian teknis berupa evaluasi performa (*benchmarking*) kecepatan, penggunaan VRAM, dan tingkat akurasi (WER dan CER) dari berbagai varian model OpenAI *Whisper*, serta pengujian non-teknis berupa evaluasi *usability* sistem oleh pengguna menggunakan kuesioner *System Usability Scale* (SUS).

## B. Tempat Penelitian

Penelitian ini dilakukan secara daring melalui pemanfaatan infrastruktur *cloud* dan perangkat keras lokal. Pengujian dilakukan menggunakan *remote pod* dari layanan *Runpod* untuk mendapatkan lingkungan komputasi GPU yang terkontrol menggunakan NVIDIA RTX A5000 dengan kapasitas VRAM sebesar 24GB. Lokasi ini dipilih karena menyediakan fleksibilitas dalam memilih spesifikasi perangkat keras yang sesuai dengan standar kebutuhan model kecerdasan buatan skala besar.

## C. Prosedur Penelitian

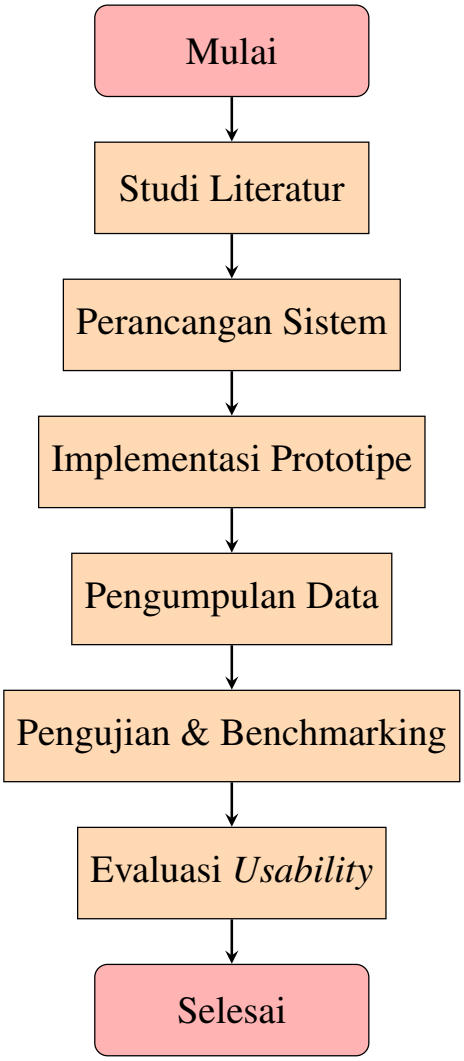
Prosedur ini dirancang secara sistematis untuk memastikan setiap tahap memiliki luaran yang dapat divalidasi pada tahap berikutnya. Berikut adalah penjabaran rinci dari setiap tahapan penelitian:

1. **Studi Literatur:** Melakukan peninjauan mendalam terhadap dokumentasi model *Whisper*, arsitektur sistem terdistribusi berbasis antrean (*queue-based*), dan metrik evaluasi ASR. Tahap ini juga mencakup eksplorasi teori *human-computer interaction* untuk mendukung perancangan antarmuka koreksi yang ergonomis. Peneliti mengumpulkan berbagai referensi ilmiah

untuk memastikan landasan teori yang digunakan telah sesuai dengan standar akademik terkini.

2. **Perancangan Sistem:** Merancang skema interaksi antara *Next.js* sebagai orkestrator tugas dan *Python Worker* sebagai unit eksekusi GPU. Perancangan meliputi skema Redis *Pub/Sub* untuk pelaporan status transkripsi secara *real-time* serta struktur data laporan hasil benchmarking. Peneliti juga merancang diagram basis data untuk menyimpan informasi pengguna, berkas audio, dan segmen transkripsi.
3. **Implementasi Prototipe:** Membangun sistem transkripsi fungsional yang mengintegrasikan seluruh komponen perangkat lunak yang telah direncanakan. *Worker* dikonfigurasi menggunakan *Faster-Whisper* untuk memaksimalkan penggunaan VRAM pada RTX A5000 melalui teknik kuantisasi yang tepat. Implementasi ini mencakup pembuatan antarmuka web interaktif yang memungkinkan pengguna untuk mengunggah audio dan memantau progres secara langsung.
4. **Pengumpulan Data:** Menyiapkan 11.540 segmen audio dari korpus *Mozilla Common Voice* beserta berkas *source-of-truth* transkrip dalam format *TSV*. Data divalidasi kesesuaiannya antara durasi audio dan teks transkrip manual untuk menghindari bias saat proses evaluasi. Peneliti memastikan keragaman audio dalam hal kebisingan dan kecepatan bicara untuk menguji ketahanan model secara objektif.
5. **Pengujian Sistem dan *Benchmarking*:** Melakukan verifikasi fungsionalitas sistem melalui *black-box testing* dan menjalankan skenario pengujian unit serta integrasi otomatis (*automated testing*) untuk mendeteksi kesalahan program. Setelah sistem dipastikan stabil, dilakukan uji coba transkripsi secara berulang untuk varian model *tiny, base, small, medium, large-v2, large-v3,* dan *large-v3-turbo*. Data yang dicatat meliputi waktu inferensi (*Real Time Factor*), penggunaan memori maksimal, dan hasil teks transkripsi mentah. Setiap pengujian dilakukan dalam lingkungan yang terkontrol untuk meminimalisir pengaruh variabel eksternal terhadap hasil pengukuran.

6. **Evaluasi *Usability*:** Melakukan pengujian kegunaan sistem menggunakan kuesioner *System Usability Scale* (SUS) terhadap 30 responden yang mewakili calon pengguna sistem (terdiri dari mahasiswa, akademisi, dan pembuat konten digital yang memiliki kebutuhan transkripsi audio). Responden diminta untuk menyelesaikan serangkaian skenario tugas pada antarmuka Skriptor, seperti mengunggah berkas audio, memantau progres transkripsi, dan melakukan koreksi teks hasil transkripsi. Setelah menyelesaikan skenario, setiap responden mengisi kuesioner SUS yang terdiri dari 10 butir pernyataan berskala Likert 5 poin. Skor SUS dihitung dan diinterpretasikan berdasarkan kategori standar (Bangor , dkk.; Brooke, 2008; 1996).




**Gambar 3.1:** Alur Penelitian (*Flowchart*)

## D. Data dan Sumber Data

Bahan penelitian utama berupa data audio dan transkrip referensi (*ground truth*).

1. **Dataset:** Menggunakan data dari korpus *Mozilla Common Voice* khusus kategori *Podcast*, yaitu *Podcast Homostoria* dan *Podcast Hari Minggoean* dalam Bahasa Indonesia. Rincian statistik dataset disajikan pada Tabel 3.1.
2. **Kriteria Audio:** Data terdiri dari rekaman percakapan alami (siniar) dengan total durasi mencakup 23,5 jam. Dataset ini dipilih karena memiliki tingkat kesulitan tinggi akibat variasi intonasi, logat lokal, dan gangguan latar

belakang (*noise*).

3. **Transkrip Referensi:** Berkas *TSV* (*tab-separated values*) digunakan sebagai *source-of-truth* yang berisi teks transkrip manual yang telah divalidasi. Berkas ini berfungsi sebagai label acuan untuk menghitung tingkat kesalahan kata dan karakter.

**Tabel 3.1:** Statistik Dataset Penelitian

| Nama Siniar | Jumlah Berkas | Jumlah Segmen | Total Durasi (Menit) |
|---|---|---|---|
| Homostoria | 16 | 5.163 | 678,02 |
| Hari Minggoean | 42 | 6.377 | 735,43 |
| **Total** | **58** | **11.540** | **1.413,45** |

## E. Alat dan Instrumen Penelitian

Instrumen penelitian terdiri dari perangkat keras dan perangkat lunak yang dipilih untuk mendukung arsitektur sistem terdistribusi berperforma tinggi. Pemilihan komponen ini didasarkan pada kebutuhan skalabilitas (*scalability*) dan efisiensi dalam menangani model AI yang berat.

1. **Perangkat Keras (Hardware):**
    - **Unit Pemrosesan AI:** NVIDIA RTX A5000 (24GB VRAM) via *Runpod* sebagai infrastruktur utama untuk tugas inferensi AI yang intensif.
    - **Server Aplikasi:** *Thin Client HP t630* dengan RAM 8GB sebagai server lokal untuk menjalankan web server *Next.js* dan *resource* lainnya yang dibutuhkan.
    - **Lingkungan Pengembangan:** Laptop *Lenovo IdeaPad Slim 3 14ALC6* dengan prosesor *AMD Ryzen 5 5500U*, RAM 20GB, dan penyimpanan *NVMe SSD* untuk proses koding, pengujian lokal, dan manajemen sistem.
2. **Perangkat Lunak (Software):**
    - **Lingkungan Pengembangan:** *Visual Studio Code* (VS Code) sebagai penyunting kode utama, *Git* untuk manajemen versi kode, dan *Bun Runtime* sebagai lingkungan eksekusi *Javascript* sisi server.

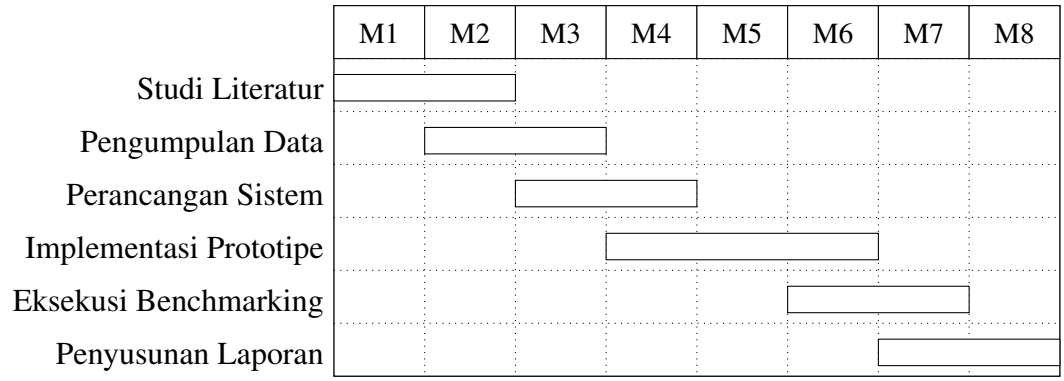
- **Bahasa Pemrograman dan Framework:** *TypeScript* untuk penulisan logika aplikasi, *Next.js* 16 sebagai kerangka kerja web utama, dan *Python* 3.10 untuk pengembangan komponen kecerdasan buatan.
- **Sistem Basis Data dan Infrastruktur:** *PostgreSQL* sebagai basis data relasional, *Redis* sebagai sistem manajemen antrean tugas, dan *Garage S3* sebagai sistem penyimpanan objek berkas audio.

3. **Instrumen Evaluasi:**
    - **Kuesioner *System Usability Scale* (SUS):** Instrumen terstandarisasi yang terdiri dari 10 butir pernyataan berskala Likert 5 poin untuk mengukur persepsi kegunaan sistem secara global (Brooke, 1996). Kuesioner ini didistribusikan kepada responden setelah menyelesaikan skenario tugas pada antarmuka Skriptor. Responden dalam pengujian ini berjumlah 30 orang yang dipilih menggunakan teknik *purposive sampling* dengan kriteria mahasiswa, akademisi, atau pembuat konten digital yang memerlukan transkripsi teks dalam aktivitasnya.

## F. Jadwal Penelitian

Penelitian direncanakan berlangsung selama delapan minggu. Rincian jadwal penelitian disajikan dalam bentuk *Gantt Chart* pada Tabel 3.1.

**Tabel 3.2:** Jadwal Penelitian (*Gantt Chart*)



# BAB IV
# HASIL DAN PEMBAHASAN

## A. Hasil Penelitian

Hasil penelitian akan disajikan di sini dalam bentuk tabel, gambar, atau grafik sesuai dengan metrik yang telah ditetapkan.

## B. Pembahasan

Interpretasi hasil dan perbandingan dengan studi terdahulu akan dijelaskan pada bagian ini.

# BAB V
# PENUTUP

## A. Kesimpulan

Kesimpulan menjawab pertanyaan penelitian secara ringkas dan padat.

## B. Saran

Rekomendasi untuk penelitian selanjutnya akan dicantumkan di sini.

# BIBLIOGRAFI


Andreyev, A. (2025, March). *Quantization for OpenAI's Whisper Models: A Comparative Analysis.* arXiv. Diakses pada 2026-04-14http://arxiv.org/abs/2503.09905 (arXiv:2503.09905 [cs]) doi: 10.48550/arXiv.2503.09905

Bain, M., Huh, J., Han, T., & Zisserman, A. (2023, August). WhisperX: Time-Accurate Speech Transcription of Long-Form Audio. In *INTERSPEECH 2023* (pp. 4489–4493). ISCA. Diakses pada 2026-04-14https://www.isca-archive.org/interspeech_2023/bain23_interspeech.html doi: 10.21437/Interspeech.2023-78

Bangor, A., Kortum, P. T., & Miller, J. T. (2008, July). An Empirical Evaluation of the System Usability Scale. *International Journal of Human–Computer Interaction*, *24*(6), 574–594. Diakses pada 2026-05-23https://www.tandfonline.com/doi/full/10.1080/10447310802205776 doi: 10.1080/10447310802205776

Brooke, J. (1996). SUS - A quick and dirty usability scale..

Chen, X., Luo, K., Gee, T., & Nejati, M. (2024, February). *Does ChatGPT and Whisper Make Humanoid Robots More Relatable?* arXiv. Diakses pada 2026-04-14http://arxiv.org/abs/2402.07095 (arXiv:2402.07095 [cs]) doi: 10.48550/arXiv.2402.07095

Christodoulidou, P., Tanner, J., Stuart-Smith, J., McAuliffe, M., Murali, M., Smith, A., … Kuschmann, A. (2025, August). A semi-automatic pipeline for transcribing and segmenting child speech. In *Interspeech 2025* (pp. 4278–4282). ISCA. Diakses pada 2026-04-14https://www.isca-archive.org/interspeech_2025/christodoulidou25_interspeech.html doi: 10.21437/Interspeech.2025-1030

Fathony, I. A. N., Mareta, A., Adiana, B. E., Wardhani, O., & Halim, A. (n.d.). Optimisasi Whisper Speech-to-Text Bahasa Indonesia dengan Hybrid Cloud dan Multi-Engine. , *18*(1).

Ferraro, A., Galli, A., La Gatta, V., & Postiglione, M. (2023, September).

Benchmarking open source and paid services for speech to text: an analysis of quality and input variety. *Front. Big Data*, *6*, 1210559. Diakses pada 2026-04-14https://www.frontiersin.org/articles/10.3389/fdata.2023.1210559/full doi: 10.3389/fdata.2023.1210559

Ferraz, T. P., Boito, M. Z., Brun, C., & Nikoulina, V. (2024, April). Multilingual DistilWhisper: Efficient Distillation of Multi-task Speech Models via Language-Specific Experts. In *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 10716–10720). Diakses pada 2026-04-14http://arxiv.org/abs/2311.01070 (arXiv:2311.01070 [cs]) doi: 10.1109/ICASSP48485.2024.10447520

Graham, C., & Roll, N. (2024, February). Evaluating OpenAI's Whisper ASR: Performance analysis across diverse accents and speaker traits. *JASA Express Letters*, *4*(2), 025206. Diakses pada 2026-04-14https://pubs.aip.org/jel/article/4/2/025206/3267247/Evaluating-OpenAI-s-Whisper-ASR-Performance doi: 10.1121/10.0024876

Imai, S., Chowdhury, T., & Stent, A. J. (n.d.). Evaluating Open-Source ASR Systems: Performance Across Diverse Audio Conditions and Error Correction Methods.

Isyanto, H., Ibrahim, W., Samsinar, R., & Sudarwati, W. (n.d.). Accurate Speech Recognition AI using Model of Deep Learning for Security Access.

Justin Joseph John, & Shailesh Satish Sharma. (2024, January). A Comparative Study of Agile and Waterfall Software Development Methodologies. *IJARSCT*, 54–57. Diakses pada 2026-04-14http://ijarsct.co.in/Paper15207.pdf doi: 10.48175/IJARSCT-15207

Latif, S., Zaidi, A., Cuayahuitl, H., Shamshad, F., Shoukat, M., Usama, M., & Qadir, J. (2025, June). *Transformers in Speech Processing: A Survey.* arXiv. Diakses pada 2026-04-14http://arxiv.org/abs/2303.11607 (arXiv:2303.11607 [cs]) doi: 10.48550/arXiv.2303.11607

Macháček, D., Dabre, R., & Bojar, O. (2023). Turning Whisper into Real-Time

Transcription System. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics: System Demonstrations* (pp. 17–24). Bali, Indonesia: Association for Computational Linguistics. Diakses pada 2026-04-14https://aclanthology.org/2023.ijcnlp-demo.3 doi: 10.18653/v1/2023.ijcnlp-demo.3

Nelson, J., & Gretkzi, M. (n.d.). Bridging the Gap Between Generalized ASR and Domain-Specific Communication Using Vosk-Driven Language Modeling.

Olev, A., & Alumäe, T. (2022). Estonian Speech Recognition and Transcription Editing Service. *BJMC*, *10*(3). Diakses pada 2026-04-14http://www.bjmc.lu.lv/fileadmin/user_upload/lu_portal/projekti/bjmc/Contents/10_3_14_Olev.pdf doi: 10.22364/bjmc.2022.10.3.14

Prabhavalkar, R., Hori, T., Sainath, T. N., Schlüter, R., & Watanabe, S. (2023, March). *End-to-End Speech Recognition: A Survey.* arXiv. Diakses pada 2026-04-14http://arxiv.org/abs/2303.03329 (arXiv:2303.03329 [eess]) doi: 10.48550/arXiv.2303.03329

Pratiwi, A. S., Wijoyo, S. H., & Wicaksono, S. A. (n.d.). Perancangan Antarmuka Pengguna Aplikasi Sistem Informasi Siswa berbasis Android menggunakan Metode User Centered Design (Studi pada : di SMK Telkom Banjarbaru).

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (n.d.). Robust Speech Recognition via Large-Scale Weak Supervision.

Riebel, J. A. (n.d.). User-centered Design Methods in Data-Intensive Software Development Processes: A State-of-the-Art Review.

Sekhar Emmanni, P. (2021, February). The Role of TypeScript in Enhancing Development with Modern JavaScript Frameworks. *IJSR*, *10*(2), 1738–1741. Diakses pada 2026-04-14https://www.ijsr.net/getabstract.php?paperid=SR24401234212 doi: 10.21275/SR24401234212

Teleki, M., Dong, X., Kim, S., & Caverlee, J. (2024, September). Comparing ASR Systems in the Context of Speech Disfluencies. In *Interspeech 2024* (pp.

4548–4552). ISCA. Diakses pada 2026-04-14https://www.isca-archive.org/interspeech_2024/teleki24_interspeech.html doi: 10.21437/Interspeech.2024-1270

Wang, S., Zhou, J., Zhao, S., & Qin, Y. (2025, June). *A Self-Training Approach for Whisper to Enhance Long Dysarthric Speech Recognition.* arXiv. Diakses pada 2026-04-14http://arxiv.org/abs/2506.22810 (arXiv:2506.22810 [cs]) doi: 10.48550/arXiv.2506.22810